1. Graph-related Concepts

Since graph theory plays an important part in CSP research, we shall define some terminologies in graph theory which we shall refer to later.

Definition 1-15:

A **graph** is a tuple (V, U) where V is a set of nodes and U ($\subseteq V \times V$) is a set of **arcs**. A node can be an object of any type and an arc is a pair of nodes. For convenience, we use *graph(G)* to denote that G is a graph.

An **undirected graph** is a tuple (V, E) where V is a set of nodes and E is a set of **edges**, each of which being a collection of exactly two elements in V. ■

The nodes in an arc are ordered whereas the nodes in an edge are not. An edge can be seen as a pair of arcs (x,y) and (y,x). A binary CSP is often visualized as a constraint graph, which is an undirected graph where the nodes represent variables and each edge represents a binary constraint.

Definition 1-16:

For all graphs (V, E), node x is **adjacent** to node y if and only if (x, y) is in E: $\forall$ graph((V, E)): ($\forall x, y \in V$: adjacent($x, y, (V, E)$) $\equiv (x, y) \in E$) ■

Definition 1-17:

A **hypergraph** is a tuple (V, E) where V is a set of nodes and E is a set of **hyperedges**, each of which is a set of nodes. For convenience we use *hypergraph((V, E))* to denote that (V, E) is a hypergraph, *hyperedges(F , V)* to denote that F is a set of hyperedges for the nodes V (i.e. F is a set of set of nodes in V), and *nodes_of(e)* to denote the nodes involved in the hyperedge e. ■

Hypergraphs are a generalization of graphs. In a hypergraph, each hyperedge may connect more than two nodes. In general, every CSP is associated with a constraint hypergraph.

Definition 1-18:

The **constraint hypergraph of a CSP** (Z, D, C) is a hypergraph in which each node represents a variable in Z, and each hyperedge represents a constraint in C. We denote the constraint hypergraph of a CSP P by $H(P)$. If P is a binary CSP and we exclude hyperedges on single nodes, then $H(P)$ is a graph.

We denote the **constraint graph of a CSP** by $G(P)$:

$\forall$ csp((Z, D, C)):

(V, E) = H((Z, D, C)) $\equiv$

(($V = Z$) $\wedge$ (E = {S | $\exists$ $c \in C \wedge S$ = variables_of(c)})) ■

What a constraint hypergraph does not show are those domains and the compound labels which are allowed or disallowed by the constraints in the CSP. Later we shall extend our definition of a *constraint graph of a CSP* to general CSPs.

Definition 1-19:

A **path** in a graph is a sequence of nodes drawn from it, where every pair of adjacent nodes in this sequence is connected by an edge (or an arc, depending on whether the graph is directed or undirected) in the graph:

$\forall$ graph((V, E)):

$\forall\ x_1, x_2, ..., x_k \in V$:

$$(\text{path}((x_1, x_2, ..., x_k), (V, E)) \equiv ((x_1, x_2) \in E) \wedge ... \wedge ((x_{k-1}, x_k) \in E)))$$

■

Definition 1-20:

A **path of length *n*** is a path which goes through $n + 1$ (not necessarily distinct) nodes:

length_of_path($(x_1, x_2, ..., x_k)$) $\equiv k - 1$ ■

Definition 1-21:

A node y is **accessible** from another node x if there exists a path from x to y:

$\forall$ graph((V, E)): ($\forall x, y \in V$: accessible(x, y, (V, E)) $\equiv$
($(x,y) \in E \vee (\exists\ z_1, z_2, ..., z_k$: path($(x, z_1, z_2, ..., z_k, y)$, (V, E))))) ■

Definition 1-22:

A graph is **connected** if there exists a path between every pair of nodes:

$\forall$ graph((V, E)):
(connected((V, E)) $\equiv$ ($\forall\ x, y \in V$: accessible(x, y, (V, E))))) ■

A constraint graph need not be connected (some variables may not be constrained, and sometimes variables may be partitioned into mutually unconstrained groups).

Definition 1-23:

A **loop** in a graph is an edge or an arc which goes from a node to itself, i.e. a loop is (x,x), where x is a node. ■

Definition 1-24:

A **network** is a graph which is connected and without loops:

$\forall$ graph((V, E)):
(network((V, E)) $\equiv$ (connected((V, E)) $\wedge$ ($\forall\ x \in V$: $(x, x) \notin E$)))) ■

Definition 1-25:

A **cycle** is a path on which end-points coincide:

$\forall$ graph((V, E)): ($\forall x_0, x_1, x_2, ..., x_k \in V$:
(cycle($(x_0, x_1, x_2, ..., x_k)$, (V, E)) $\equiv$
(path($(x_0, x_1, x_2, ..., x_k)$, (V, E)) $\wedge$ $x_0 = x_k$))) ■

Definition 1-26:

An **acyclic graph** is a graph which has no cycles:

$\forall$ graph(G): (acyclic(G) $\equiv$ ($\neg \exists$ path(p, G): cycle(p, G))) ■

Definition 1-27:

A **tree** is a connected acyclic graph:

$\forall$ graph(G): (tree(G) $\equiv$ (network(G) $\wedge$ ($\neg \exists$ path(p, G): cycle(p, G)))) ■

Definition 1-28:

A binary relation (<) on a set S is called an **ordering** of S when it is *irreflexive*, *asymmetric* and *transitive*:

irreflexive(S, <): $\forall\, x \in S: \neg\, x < x$
asymmetric(S, <): $\forall\, x, y \in S: (x < y \Rightarrow \neg\, y < x)$
transitive(S, <): $\forall\, x, y, z \in S: (x < y \wedge y < z \Rightarrow x < z)$. ■

Definition 1-29:

A set S is **totally ordered** if every two elements in S are ordered. Such an ordering is called a **total ordering** of the elements in S:

total_ordering(S, <) ≡ ($\forall\, x, y \in S: x < y \vee y < x$). ■

2. More Graph-related Concepts

Every binary CSP is associated with a constraint graph. Many CSP solving techniques are designed to exploit the topology of the constraint graphs of the problems. We shall define the relevant concepts in graph theory.

In this section, we shall continue to denote graphs by G, where $G = (V, E)$, with V being a set of nodes and E being a set of edges (see Definition 1-15). All graphs referred to in this section are undirected graphs without loops.

The first group of definitions are about the width of a graph. These concepts are useful for explaining the ordering of variables in searching.

Definition 3-20:

Given a graph (V, E) and a total ordering on its nodes, the **width of a node** v is the number of nodes that are before and adjacent to v:

$\forall$ graph$((V, E))$: ($\forall$ <: total_ordering$(V, <)$:
    ($\forall\ x \in V$: width$(x, (V, E), <) \equiv |\{y \mid y < x \wedge (x, y) \in E\}|$)) ■

Definition 3-21:

The **width of a graph under an ordering** is the maximum width of all the nodes in the in the graph under that ordering:

$\forall$ graph$((V, E))$: ($\forall$ <: total_ordering$(V, <)$:
    width$((V, E), <) \equiv$ MAX width$(x, (V, E), <)$: $x \in V$) ■

Definition 3-22:

The **width of a graph** is the minimum width of the graph under all possible orderings of its nodes:

$\forall$ graph((V, E)): width((V, E)) $\equiv$ MIN width((V, E), <): total_ordering(V, <) ■

For example, Figure 3.5(a) shows a graph.

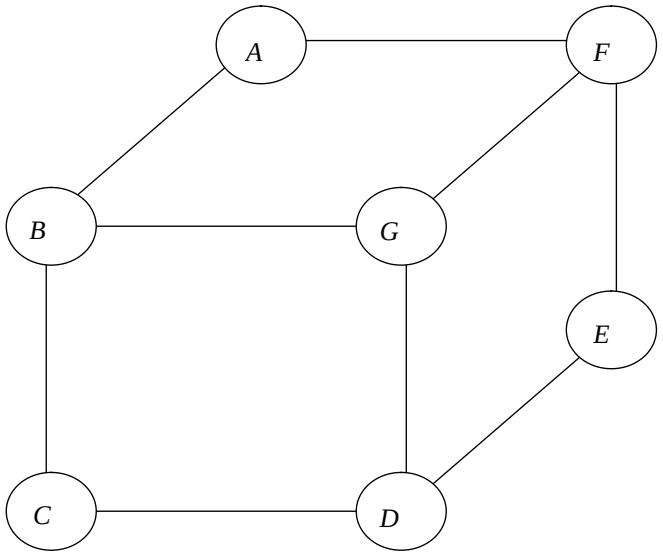
If the ordering of the nodes is (A, B, C, D, E, F, G), then the width of the nodes are 0, 1, 1, 1, 1, 2, 3, respectively (Figure 3.5(b)). Therefore the width of this ordering is 3, which is the maximum width among all nodes.

Should the ordering be (G, F, E, D, C, B, A), the width of the nodes would be 0, 1, 1, 2, 1, 2, 2 (Figure 3.5(c)). The width of this ordering is 2.

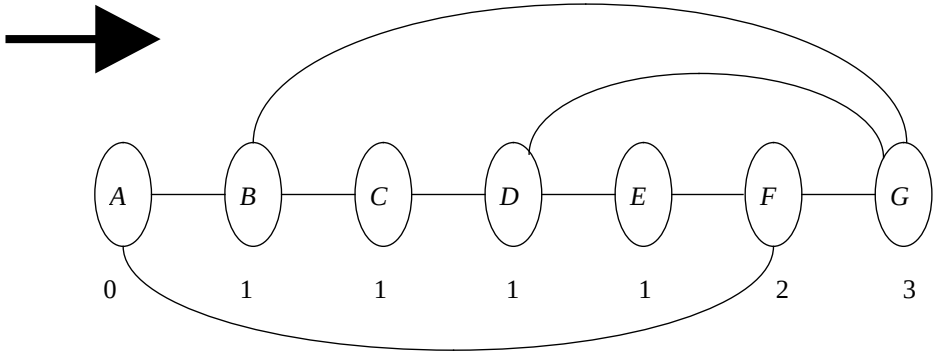
Definition 3-23:

A graph $G' = (V', E')$ is **induced by** another graph $G = (V, E)$ if V' is a subset of V, and E' is the set of all the edges in E which join the nodes in V':

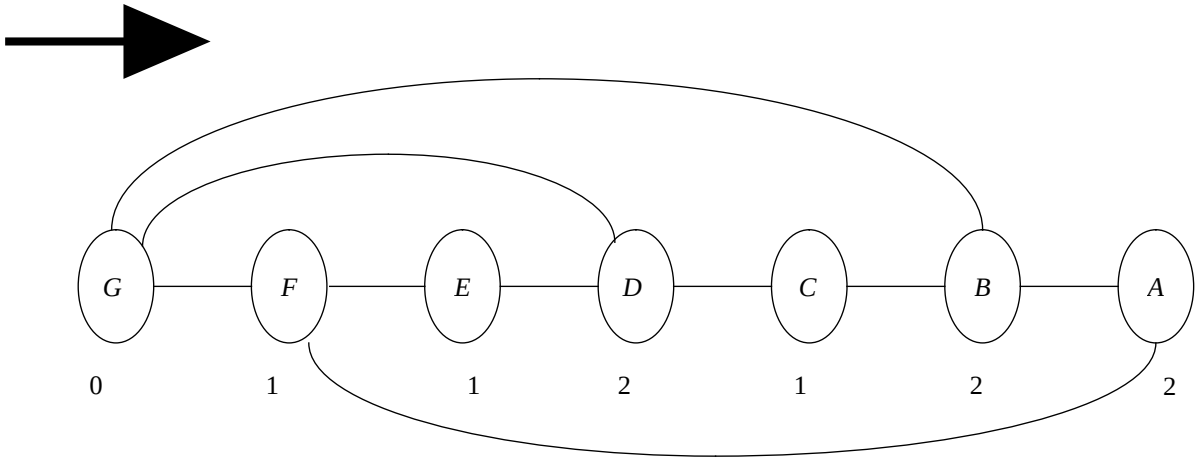
$\forall$ graph((V, E)), graph((V', E')): induced_by((V', E'), (V, E)) $\equiv (V' \subseteq V) \wedge (E' = \{(a, b) \mid (a, b) \in E \wedge a \in V' \wedge b \in V'\})$ ■



(a) A constraint graph to be labelled



(b) Width of the nodes given the order *A, B, C, D, E, F, G*



(c) Width of the nodes given the order *G*, *F*, *E*, *D*, *C*, *B*, *A*

Figure 3.5 Example of a constraint graph with the width of different orderings shown

Definition 3-24:

The **neighbourhood** of a node v in a graph G is the set of all the nodes in G which are adjacent to v:

$\forall$ graph$((V, E))$: $(\forall\ v \in V$: neighbourhood$(v, (V, E)\ ) \equiv \{w \mid w \in V \wedge (v, w) \in E\})$ ■

Definition 3-25:

The **degree (which is sometimes called valency in the literature) of a node** in a graph is the number of nodes to which this node is adjacent:

$\forall$ graph$((V, E))$: $(\forall\ v \in V$: degree$(v, (V, E)) \equiv |$neighbourhood$(v, (V, E))|)$ ■

Definition 3-26:

A ***k*-tree** is either a complete graph with k nodes, or a graph in which one can find a node v that satisfies three conditions:

(1) that it is adjacent to k nodes;
(2) its neighbourhood (which has k nodes) forms a complete graph;
(3) the graph without both v and the edges involving v forms a k-tree.

A k-tree which is a complete graph G with k nodes is called a **trivial *k*-tree**, denoted *trivial_k-tree*(G):

$$\forall\ \text{graph}((V, E)): k\text{-tree}((V, E)) \equiv$$
$$((|V| = k \wedge \text{complete_graph}((V, E))) \vee$$
$$\exists\ v \in V: (\text{degree}(v, (V, E)) = k \wedge$$
$$(G' = (\text{neighbourhood}(v, (V, E)), E') \Rightarrow \text{induced_by}(G', (V, E)) \wedge$$
$$\text{complete_graph}(G')) \wedge$$
$$(k\text{-tree}((V - \{v\}, E - \{(v, w) \mid (v, w) \in E\}))))) \blacksquare$$

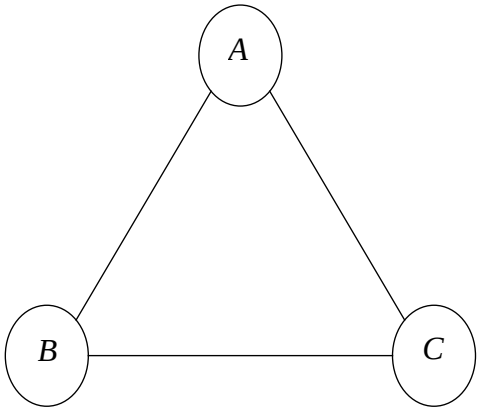
Figure 3.6 shows examples and counter-examples of k-trees.

The graph in Figure 3.6(a) is a trivial 3-tree which is a complete graph with 3 nodes.

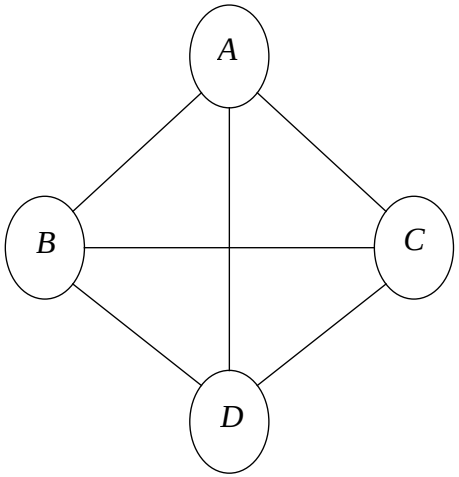
The graph in Figure 3.6(b) is the graph in Figure 3.6(a) with an extra node D added. It is a 3-tree because there exists a node D which has a degree of 3, its neighbourhood $\{A, B, C\}$ forms a complete graph and the graph without D is a (trivial) 3-tree.

In each of the graphs in Figures 3.6(c) and (d), one more node is added. They are 3-trees as the added nodes satisfy the above conditions.

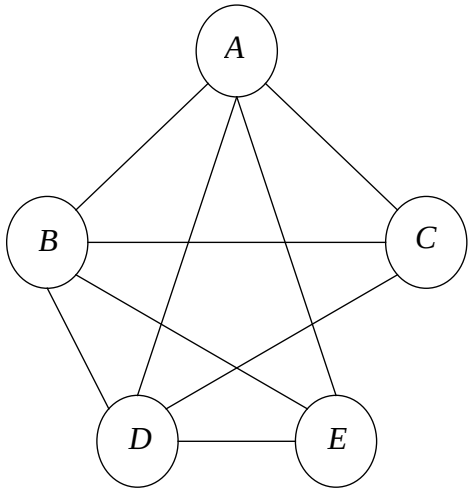
The graph in Figure 3.6(e) is not a 3-tree because there exists only one node, F, which degree is 3. But the neighbourhood of F, which is the set $\{C, D, E\}$, does not form a complete graph.



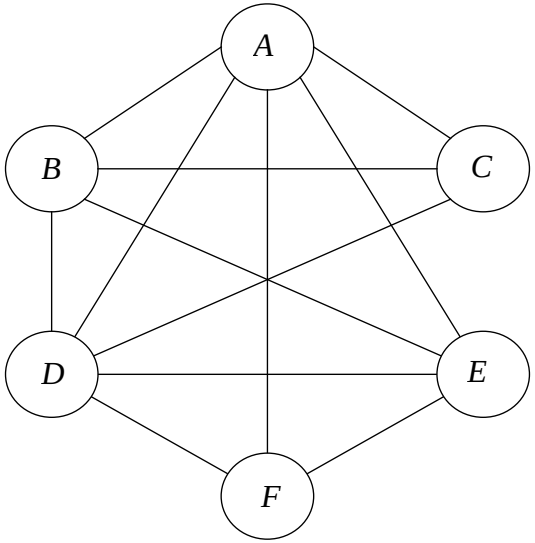
(a) A (trivial) 3-tree



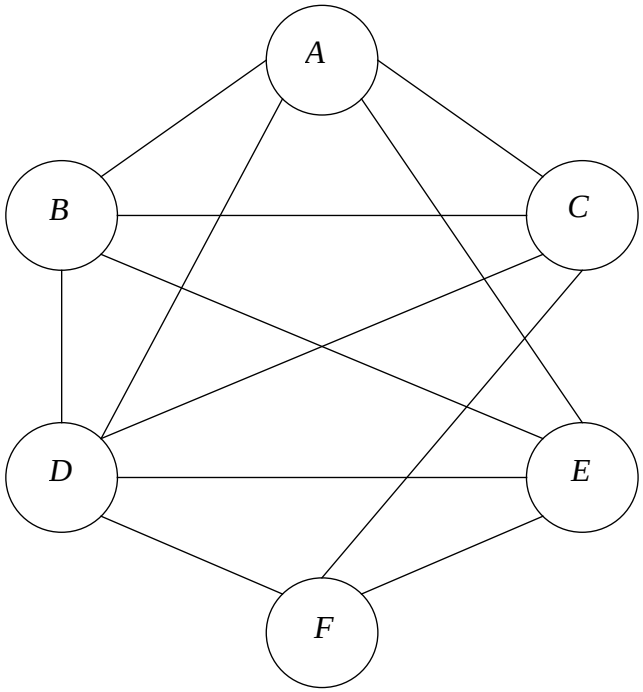
(b) A 3-tree with 4 nodes, which is also a trivial 4-tree



(c) A 3-tree with 5 nodes



(d) A 3-tree with 6 nodes



(e) A graph which is NOT a 3-tree (*F* is the only node with degree=3, but the neighbourhood of F, {*C*,*D*,*E*} is not a complete graph)

Figure 3.6 Examples and counter-examples of k-trees

Definition 3-27:

G' = (V', E') is a **partial graph** of G = (V, E) if V' is a subset of V and E' is a subset of the edges in E which join the nodes in V'. *Partial_graph(G', G)* reads "G' is a partial graph of G":

$\forall$ graph((V, E)), graph((V', E')):

partial_graph((V', E'), (V, E)) $\equiv$

$(V' \subseteq V \wedge E' \subseteq \{(a, b) \mid a \in V' \wedge b \in V' \wedge (a, b) \in E\})$ ■

When E' equals the set of *all* edges in E which join the nodes in V', G' is induced by G.

Definition 3-28:

Graph G **embeds** graph G' if G' is a partial graph of G and G is a k-tree for some integer k. *Embedding(G, G')* reads "G embeds G'":

$\forall$ graph(G), graph(G'): embedding(G, G') $\equiv$

(partial_graph(G', G) ∧ (∃ k: k-tree(G))) ■

Definition 3-29:

G is a **partial-k-tree** if there exists a k-tree G' of which G is a partial graph:

∀ graph(G): ∀ k: partial-k-tree(G) ≡ (∃ G': partial_graph(G, G') ∧ k-tree(G')) ■

According to this definition, any graph is a partial-k-tree for a sufficiently large k.

Definition 3-30:

A **weak-k-tree** is either a complete graph with k or less nodes, or a graph in which one can find a node v which satisfies three conditions:

(1) that it is adjacent to no more than k nodes;
(2) its neighbourhood forms a complete graph;
(3) the graph without v and edges involving v forms a weak-$k$tree:

$\forall$ graph((V, E)):
  weak-k-tree((V, E)) $\equiv$
    (($|V| \leq k \wedge$ complete((V, E))) $\vee$
      $\exists\, v \in V$: (degree(v, (V, E)) $\leq k \wedge$
          (G'=(neighbourhood(v, (V, E)), E') $\Rightarrow$ induced_by(G', (V, E)) $\wedge$
            complete(G')) $\wedge$
              (weak-k-tree($(V - \{v\}, E - \{(v, w) \mid (v, w) \in E\})$)))) ■

The definition of **weak-k-tree** is similar to k-tree except that all "$= k$" are replaced by "$\leq k$".